22.05.2019

EE 241 DIGITAL CIRCUITS
FINAL SOLUTION MANUAL

Name:

Q1) (20 points) Form the Verilog module to calculate the factorial of a given eight-bit number.

Solution

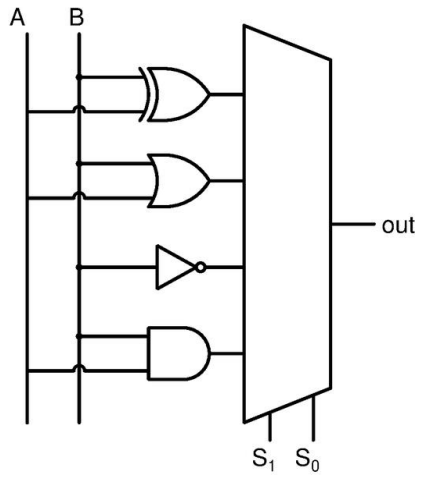
```
module factorial(number, result, clk, count);

        input clk;
        input [7:0] number;
        output reg [31:0] result;
        output reg [7:0] count;

        initial
          begin
          result=32'b1;
          count=8'd0;
          end

        always@(posedge clk)
        begin
                if (count<number)
                begin
                count=count+1;
                result=result*count;
                end
        end
endmodule
```

Q2) (20 points) Simplified logic schematic of an opcode fetch circuit in an arithmetic-logic unit (ALU) is shown below.



a) Form the Verilog description of the circuit using dataflow modeling.

Solution

```
module q2(A, B, S1, S0, out);
input A, B, S1, S0;
output out;
assign out = (~S1&~S0&(A^B)) | (~S1&S0&(A|B)) | (S1&~S0&(~B)) | (S1&S0&A&B);
endmodule
```

b) Modify the module such that “A”, “B” and “out” are represented in eight bits. Here, only form the Verilog description using behavioral modeling.

Solution

```
module q2(A,B,S1,S0,out);
input [7:0] A,B;
input S1,S0;
output reg [7:0] out;
always @(*)
case({S1,S0})
0: out=(A^B);
1: out=(A|B);
2: out=~B;
3: out=A&B;
endcase
endmodule
```

Q3) (30 points) You are asked to implement the temperature display unit using the Basys-3 Board. Minimum and maximum numbers that can be displayed are 0 and 99, respectively. To make the system more modular, represent subparts of the module with below submodules.

a) If btnC is pressed, the temperature will be displayed in Fahrenheit. Otherwise, it will be displayed in Celsius degrees. Form a Verilog module that selects either Celsius or Fahrenheit to be displayed with the press of btnC. Layout of the module is as follows.

```
module  select(Celsius, Fahrenheit, btnC, result);

input [6:0] Celsius, Fahrenheit;
input btnC;
output reg [6:0] result;
always @(*)
   if(btnC)
      result=Fahrenheit;
   else
      result=Celsius;

endmodule
```

b) Data coming from the temperature sensor is in Celsius degrees. Therefore, it needs to be converted to Fahrenheit. Form a Verilog module that converts the temperature value from Celsius to Fahrenheit using the following formula.

$$\frac{celcius}{100} = \frac{fahrenheit - 32}{180}$$

Layout of the module is as follows.

```
module convert (Celsius, Fahrenheit);

input [6:0] Celsius;
output [6:0] Fahrenheit;
assign Fahrenheit=(Celsius*180)/100+32;

endmodule
```

c) The temperature will be displayed on the 7-segment display. Therefore, a 7-segment converter module is needed. Form a Verilog module that takes an input between 0 - 9 and calculates the corresponding 7-segment outputs. Layout of the module is as follows.

```
module sevseg(in, out);

        input [3:0] in;
        output reg [6:0] out;
        always @(*)
          case(in)
```

```
            0:  out = 7'b1000000; // 0
            1:  out = 7'b1111001; // 1
            2:  out = 7'b0100100; // 2
            3:  out = 7'b0110000; // 3
            4:  out = 7'b0011001; // 4
            5:  out = 7'b0010010; // 5
            6:  out = 7'b0000010; // 6
            7:  out = 7'b1111000; // 7
            8:  out = 7'b0000000; // 8
            9:  out = 7'b0010000; // 9
          endcase

endmodule
```

d) There is an upper limit for the clock frequency while performing the scanning operation on the two digits of the 7-segment display. Form a module which divides the master clock of the Basys-3 Board (100 MHz) to **500** Hz. Layout of the module is as follows.

```
module clkdiv(clk, clk2);

          input clk;
          output reg clk2;
          reg[16:0] A;
          always @(posedge clk)
          if (A==17'd99999)
             begin A<=17'd0; clk2 <= ~clk2; end
           else A<=A+1'b1;

endmodule
```

e) Combine all submodules defined above and create a state machine for displaying tens and ones digit of the temperature value. Temperature sensor is implemented with sw[6:0]. Layout of the module is as follows.

```
module main(clk, btnC, sw, seg, an);

          input clk,btnC;
          input [6:0] sw;
          output [6:0] seg;
          output reg [3:0] an;
          wire [6:0] fahrenheit,result;
          wire clk2;
          reg [3:0] segin;
          reg state=1'b0;

          convert u1(sw,fahrenheit);
          select u2(sw,fahrenheit,btnC,result);
          clkdiv u3(clk,clk2);
          sevseg(segin,seg);
          always @(posedge clk2)
          begin
```

```
            state <= ~state;
               case(state)
               0:begin segin=result/10; an=4'b1101; end
               1:begin segin=result%10; an=4'b1110; end
               endcase
            end

endmodule
```

Q4) (30 points) You are asked to develop a TV remote control unit using the Basys-3 Board. To make the system more modular, represent each part of the problem with a submodule as given below.

a) btnU and btnD changes the channel to be watched. Channel numbers are between 0 and 9. The channel number will be displayed on the rightmost 7-segment display. Increasing channel number after 9 will reset it to 0. Likewise, decreasing channel number below 0 will set it to 9. All button-press durations are assumed to be less than one clock period. Layout of the module is as follows.

```
module channel(clk, btnU, btnD, seg, an);

        input clk,btnU,btnD;
        output [6:0] seg;
        output [3:0] an;
        wire U,D;
        reg [3:0] segin=0;
        assign an=4'b1110;
        edgedetect u1(clk,btnU,U);
        edgedetect u2(clk,btnD,D);
        segment u5(segin,seg);
        always @(posedge clk)
          case({U,D})
          1: if(segin==0) segin<=9; else segin<=segin-1;
          2: if(segin==9) segin<=0; else segin<=segin+1;
          endcase
        endmodule

        module segment(in,out);
          input [3:0] in;
          output reg [6:0] out;
          always @(in)
               case(in)
               0:out <= 7'b1000000;
               1:out <= 7'b1111001;
               2:out <= 7'b0100100;
               3:out <= 7'b0110000;
               4:out <= 7'b0011001;
               5:out <= 7'b0010010;
               6:out <= 7'b0000010;
               7:out <= 7'b1111000;
               8:out <= 7'b0000000;
               9:out <= 7'b0010000;
               endcase

endmodule
```

b) btnL and btnR control the volume of the TV. The volume level is between 0 and 16 and it will be displayed on led [15:0]. Increasing the volume after 16 will not change anything. Likewise, decreasing the volume below 0 will not change anything. All button-press durations are assumed to be less than one clock period. Layout of the module is as follows.

```
module volume (clk, btnL, btnR, led);

        input clk,btnL,btnR;
        output reg [15:0] led;
        wire L,R;
        edgedetect u1(clk,btnL,L);
        edgedetect u2(clk,btnR,R);
        always @(posedge clk)
          case({L,R})
          1: if(led==16'd0) led<=16'd0; else led<=led/2;
          2: if(led==16'hFFFF) led<=16'hFFFF; else led<=(2*led)+1;
          endcase
endmodule
```

Extra:

```
module edgedetect(clk,in,out);
input in,clk;
output reg out;
wire x;
reg r1,r2;
assign x=in&(r1^r2);
always @(posedge clk)
begin
r1 <= in;
r2 <= r1;
end

always @(negedge clk)
out <= x;
endmodule
```

c) Form the main module for instantiating the submodules above. Layout of this main module is as follows.

```
module main(clk, btnL, btnR, btnU, btnD, seg, an, led);
        input btnL,btnR,btnU,btnD,clk;
        output [6:0] seg;
        output [3:0] an;
        output [15:0] led;

        channel u1(clk,btnU,btnD,seg,an);
        volume u2(clk,btnL,btnR,led);
        endmodule
```